

1 Key Concepts

1.1 Proof Techniques

Direct proof. This is used for proving *implications*, i.e., statements of the form $P \implies Q$ (i.e., “if P , then Q ”). We assume that P is true, then write a sequence of logical consequences *directly* showing that Q must also be true.

Proof by contrapositive. This is also used for proving implications $P \implies Q$. But in this case we directly prove the logically equivalent *contrapositive* of the statement, which is $\neg Q \implies \neg P$. That is, we assume that $\neg Q$ is true, then derive $\neg P$ as a logical consequence.

Proof by contradiction. Suppose we want to prove a statement P . We can do this by first assuming that P is false, then deriving some logical contradiction. Since the assumption that P is false led to a contradiction, it must be that P is true.

1.2 Induction

Induction is used to prove a statement about a countably infinite set, such as the positive integers. To prove a statement $P(n)$ about the positive integers, we show two things:

- (1) $P(1)$ is true.
- (2) If k is any positive integer and $P(k)$ is true, then $P(k+1)$ is true as well.

If we can show these two things, then we can conclude from (1) that $P(1)$ is true. Then, using (1) and (2) together, we conclude that $P(2)$ is true, and then so is $P(3)$, and so on.

Strong Induction Strong induction differs from typical induction in that, during the inductive step, typical induction proves a claim of the form $P(k) \rightarrow P(k+1)$ whereas strong induction proves a claim of the form $P(1) \wedge P(2) \wedge \dots \wedge P(k) \rightarrow P(k+1)$.

1.3 Asymptotic Notation

In the following, let $f(n)$ and $g(n)$ be positive functions. We define the following asymptotic notations.

Definition 1. We say that $f(n) = O(g(n))$ if there exist positive constants $c, n_0 > 0$ such that $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$.

Definition 2. We say that $f(n) = \Omega(g(n))$ if there exist positive constants $c, n_0 > 0$ such that $f(n) \geq c \cdot g(n)$ for all $n \geq n_0$.

(Notice that the only difference between [Definitions 1](#) and [2](#) is the $\leq$ versus the $\geq$ comparison.)

Definition 3. We say that $f(n) = \Theta(g(n))$ if $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$.

Lemma 4. When $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$ exists or is infinite, $f(n) = O(g(n))$ if and only if the limit is finite.

Note: it is not *necessary* that the limit exists: even if it does not, it may still be the case that $f(n) = O(g(n))$.

Lemma 5. When $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$ exists or is infinite, $f(n) = \Omega(g(n))$ if and only if the limit is greater than zero (including infinite).

Again, it is not *necessary* that the limit exists: even if it does not, it may still be the case that $f(n) = \Omega(g(n))$.

1.4 The Potential Method

The potential method is a useful technique to reason about the amount of time an algorithm takes before halting. By defining an appropriate potential function for an algorithm, and establishing how fast it decreases and a lower bound for it, we can get an upper bound on the number of “units of work” the algorithm performs (as a function of its input).

Formally, we define a function $s: A \rightarrow \mathbb{R}$ that maps the “state” of the algorithm (i.e., its internal variables) to a (finite) real number. The function s is a valid potential function if:

1. s decreases by some *fixed positive constant* $C > 0$ with every “unit of work” done by the algorithm—for a suitable notion of “work unit”, e.g., a loop iteration;
2. s has some *lower bound* B , i.e., the algorithm is guaranteed to terminate once $s \leq B$.

The first condition ensures that $s \leq B$ will eventually occur within a finite number of work units: specifically, $(I - B)/C$ units, where I is the initial value of the potential function (for the original input to the algorithm). The second condition then ensures that the algorithm will *terminate* within that number of work units.

2 Practice Problems

1. Comparing asymptotic running times.

On the same computer with the same input, an algorithm having $O(n \log n)$ worst-case running time runs in less time than **all/some/no** algorithms having $O(n^2)$ worst-case running time.

2. Asymptotics.

For the following pairs of functions $f(n)$ and $g(n)$, is $f(n) = O(g(n))$? Justify your answer by applying the definition of big-O or by applying a limit argument.

- $f(n) = \log_a n$, $g(n) = \log_b n$ for constants $a, b > 1$.

- $f(n) = 4^n$, $g(n) = 2^n$

3. Faulty proofs.

For each of the proposed “proofs” of the following (true!) statements, determine what is incorrect about the proof.

- (a) Claim: There is no even prime greater than 2. Proof: 3 is a prime that is greater than 2 and it is not even, so there must be no even prime greater than 2.

- (b) Claim: If n^2 is even, then n is even. Proof: If n is even, then $n = 2k$ for some integer k . Therefore, $n^2 = 4k^2 = 2 \cdot 2k^2$ is even.

- (c) Claim: The sum of the first i positive odd integers equals the i th square. Proof: $1 = 1^2$, and $1 + 3 = 4 = 2^2$, and $1 + 3 + 5 = 9 = 3^2$, and so on. The pattern will continue, so the claim is proven.

- (d) Claim: 37 is the largest prime smaller than 41. Proof: 37 is less than 41, and is prime since its only divisors are 1 and 37.

- (e) Claim: The sum of two even integers is even. Proof: Let n, m be any two even integers. Since they are both even, then let $n = 2k$ and $m = 2k$ for some integer k . $n + m = 2k + 2k = 4k = 2(2k)$, therefore the sum is also even.

4. **Proof by contradiction.**

Prove by contradiction that $\sqrt{7}$ is irrational.

5. **Euclid's algorithm.**

Which pair of numbers takes the greatest number of iterations to find the GCD when using Euclid's algorithm?

- ☐ (800, 300)
- ☐ (100, 55)
- ☐ (29, 18)
- ☐ (1000, 999)
- ☐ (19, 11)

6. Induction

Let $a_1, a_1r, a_1r^2, \dots, a_1r^n$ be a finite geometric sequence with common ratio r and initial term a_1 (notice the 1-indexing). Prove using induction that the sum of the i terms is equal to $S_i = \frac{a_1(1-r^i)}{1-r}$.

7. Strong Induction

Suppose there is a pile of n stones that must be split into n piles of 1 stone each. Each split is characterized by taking a pile with > 1 stone and splitting the pile into 2 smaller piles each having at least 1 stone. Prove that no matter how the splits are performed over an initial pile of size n (i.e. any ordering, equal/unequal splits, etc.), the sum of the pairwise products of the splits will equal $\frac{n(n-1)}{2}$.

As an example of the process, suppose there are 4 stones to start. One possible way to perform the splits is to have 3 splits as follows: $(2 - 2), (1 - 1) - 2, 1 - 1 - (1 - 1)$. This notation is intended to indicate first splitting into 2 piles of 2, then splitting 1 of the 2 piles of 2 into 2 piles of 1, and finally splitting the other pile of 2 into 2 piles of 1. Computing the sum of the pairwise products of the splits, we have $2 \cdot 2 = 4$ from the first split since the first split resulted in 2 piles of 2, then $1 \cdot 1 = 1$ from the second split since the second split resulted in 2 piles of size 1, and finally $1 \cdot 1 = 1$ for the third split. Summed together, we have $4 + 1 + 1 = 6 = \frac{4(4-1)}{2}$. The same process can be done by instead having the first split be $(3 - 1)$ and so on and the pairwise sum of splits will still be 6 (you can check!).

8. Potential functions.

Suppose *input* is a function that returns a user-specified positive integer. For each of the following problems, do the following tasks:

- Determine whether the following program halts for any sequence of (valid) inputs.
- If so, provide a proof using the potential method; if not, describe a sequence of inputs that causes the program to never halt.

(a)

```
1:  $x \leftarrow \text{input}()$ 
2:  $y \leftarrow \text{input}()$ 
3: while  $x > 0$  and  $y > 0$  do
4:    $z \leftarrow \text{input}()$ 
5:   if  $z$  is even then
6:      $x \leftarrow x - 1$ 
7:      $y \leftarrow y + 1$ 
8:   else
9:      $y \leftarrow y - 1$ 
```

(b)

```
1:  $x \leftarrow \text{input}()$ 
2:  $y \leftarrow \text{input}()$ 
3: while  $x > 0$  and  $y > 0$  do
4:    $z \leftarrow \text{input}()$ 
5:   if  $z$  is even then
6:      $x \leftarrow x - 1$ 
7:      $y \leftarrow y + 1$ 
8:   else
9:      $y \leftarrow y - 1$ 
10:     $x \leftarrow x + 1$  // This line differs from the program in (a)
```

(c)

```
1:  $x \leftarrow \text{input}()$ 
2:  $y \leftarrow \text{input}()$ 
3: while  $x > 0$  or  $y > 0$  do // This line differs from the program in (a)
4:    $z \leftarrow \text{input}()$ 
5:   if  $z$  is even then
6:      $x \leftarrow x - 1$ 
7:      $y \leftarrow y + 1$ 
8:   else
9:      $y \leftarrow y - 1$ 
```